IMU-Controlled Mobile Robot – Final Report

School of Engineering and Natural Sciences
Electrical and Electronics Engineering – Control Systems

Group Members:
- Mohammed SharafAldeen (Team Leader) – 61220027
- Nazende Yağmur Uysal – 10200101

1. Abstract

This project presents the design, modeling, and control of a mobile robot system driven by two DC motors and controlled wirelessly through an IMU-based transmitter. The robot's movement is governed by tilt gestures captured via the MPU9250 sensor, which detects pitch and roll angles and maps them to directional commands—forward, backward, left, and right. Encoder feedback from the motors is used to monitor and calculate the rotational speed (RPM), allowing for real-time performance evaluation. The system is modeled using physical motor parameters to derive transfer functions for both linear and angular velocities. MATLAB and Simulink are used to analyze the system's dynamic behavior through step responses, Bode plots, and root locus diagrams. PID controllers are designed and tuned to improve system stability, reduce steady-state error, and enhance responsiveness. The closed-loop behavior is validated both in simulation and through experimental testing using encoder outputs. Overall, the project demonstrates an effective integration of control theory, signal processing, embedded programming, and wireless communication for intelligent motion control of mobile robotic platforms.

2. Introduction and Motivation

This project aims to design and implement a wireless-controlled mobile robot that responds to the user's hand or body movements. By utilizing an MPU9250 inertial measurement unit (IMU), the system captures real-time orientation data—specifically, pitch and roll—which are then used to determine movement direction and speed.

The control strategy is implemented using two Arduino UNO boards, one acting as a transmitter and the other as a receiver. The transmitter processes the IMU data and sends movement commands wirelessly using APC220 modules. On the receiver side, the robot interprets these commands to control two DC motors through an L298N motor driver. A 3D-printed chassis provides structural support, while a dedicated battery pack supplies power to the motors and control electronics.

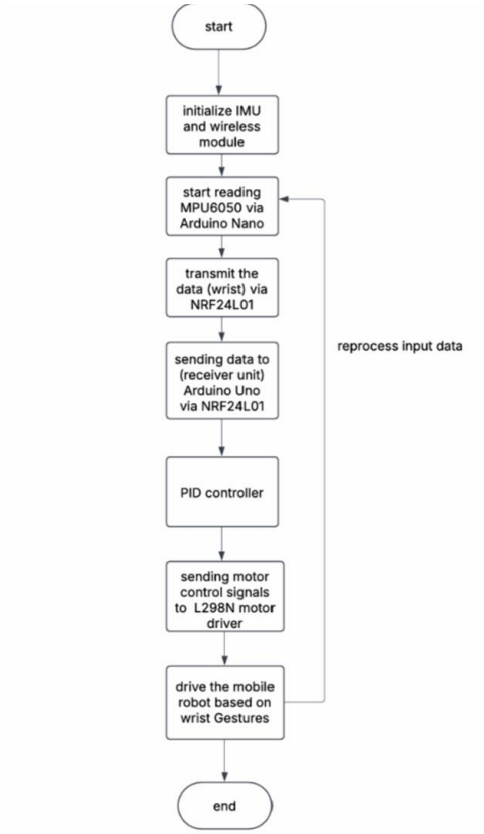
The system incorporates both open-loop and closed-loop control strategies. Rotary encoders attached to the motors allow feedback-based speed regulation via PID control, ensuring smoother and more accurate motion. Through this project, the effectiveness of IMU-based direction control, real-time wireless communication, and feedback stabilization in mobile robotic systems is explored, tested, and analyzed using tools such as step response analysis and Bode plots techniques.

3. Task Plan:

Task/Phase	Status	Team Members	Explanation & Details
Component Selection & Procurement	Completed	Mohammed, Nazende	Selection and gathering of essential components including MPU9250, Arduino, APC220, etc.
Wireless Communication Setup	Completed	Mohammed	Establishing serial communication between two Arduino boards using APC220 modules.
IMU Sensor Integration	Completed	Mohammed	Connecting and programming MPU9250 for roll and pitch measurement, including smoothing.
Motion Control Logic Development	Completed	Mohammed	Writing Arduino code to control motor directions (forward, backward, left, right) based on IMU tilt.
PWM Speed Control Implementation	Completed	Mohammed	Mapping tilt angles to PWM values for proportional speed control of the motors.
Mechanical Assembly	Completed	Nazende	Mounting motors, wheels, and assembling the 3D-printed chassis and battery pack.

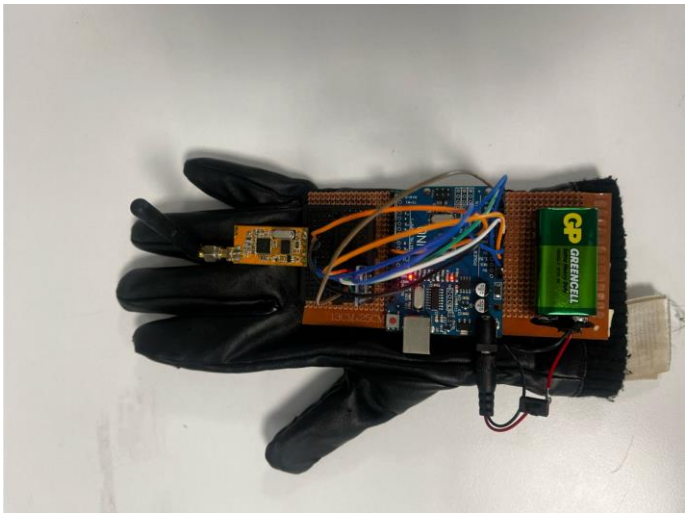
Motor Driver Integration	Completed	Nazende	Connecting and wiring the L298N motor driver to the motors and Arduino.
Data Acquisition and RPM Monitoring	Completed	Mohammed	Using encoder feedback to compute motor RPM for performance monitoring.
System Modeling in MATLAB/Simulink	Completed	Mohammed	Deriving transfer functions and simulating step responses, root locus, and Bode diagrams.
PID Controller Design and Tuning	Completed	Mohammed	Implementing PID control for both linear and angular velocity and tuning it using MATLAB.
System Testing	Completed	Mohammed, Nazende	Performing tests to validate the behavior of the robot under different tilt inputs.
Final Reporting and Documentation	Completed	Mohammed, Nazende	Preparing all figures, plots, codes, and written sections for the final report.

4. System Overview

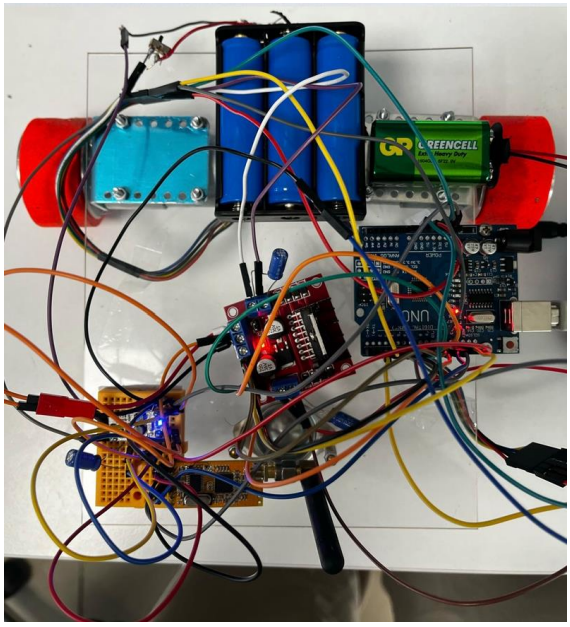


The system consists of two major units:

1. Transmitter Unit (Wrist): IMU sensor and transmitter Arduino



2. Receiver Unit (Robot): Motor driver, DC motors, and receiver Arduino



5. Materials and Components

Component	Purpose
MPU9250	Measures tilt (acceleration + gyro)
Arduino UNO (2 modules)	Data acquisition, processing, control logic
APC220 (2 modules)	Wireless communication
L298N Motor Driver	Controls motor direction and speed
2× DC Motors + Wheels	Robot propulsion
3D-Printed Chassis	Robot body, designed in Fusion 360
Battery Pack	Power source for motors and controller

6. Methodology

The development of the wireless-controlled mobile robot was carried out through a systematic approach involving hardware integration, signal processing, and control algorithm implementation. The primary goal was to control the movement of a two-wheeled robot using body tilt detected by an MPU9250 inertial measurement unit (IMU). The IMU measured real-time pitch and roll angles, which were processed to generate direction and speed commands based on predefined thresholds and mapping functions.

Two Arduino UNO modules were used in this setup—one on the transmitter side and another on the receiver side. The transmitter module acquired orientation data from the MPU9250, processed it to determine the desired direction (FORWARD, BACKWARD, LEFT, RIGHT), and

mapped the tilt to a PWM speed range. This control information was then wirelessly transmitted to the receiver Arduino using a pair of APC220 modules.

On the receiver side, the Arduino decoded the incoming commands and controlled the L298N motor driver accordingly. The L298N module regulated the direction and speed of two DC motors, propelling the robot as per the instructions received. A 3D-printed chassis was used to house all components, ensuring proper balance and mechanical structure. The entire system was powered using a dedicated battery pack that supplied voltage to both the motors and microcontrollers.

To enhance performance and ensure stability, PID control was implemented on the receiver Arduino. Feedback from rotary encoders was used to calculate real-time motor speeds, which were then compared to the desired speeds to adjust PWM output accordingly. Throughout the process, system behavior was analyzed using tools such as step response plots, Bode diagrams, and root locus graphs. These tools helped fine-tune the gains and evaluate both open-loop and closed-loop dynamics of the system

7. Equations of motion:

Parameters:

The torque constant Kt=0.5 Nm/A indicates that the motor produces 0.5 newton-meters of torque for every ampere of current supplied. This directly affects the motor's ability to generate mechanical force. The back EMF constant Ke=0.5 V/(rad/s) signifies the voltage generated by the motor when it rotates, which opposes the applied voltage and introduces damping. The armature resistance Ra=1.2 Ω accounts for energy losses in the form of heat within the motor windings, impacting current flow and therefore torque.

On the mechanical side, the moment of inertia J=0.5 kgM^2 characterizes the motor's resistance to changes in rotational speed, affecting the acceleration response. The viscous damping coefficient b=0.5 N models friction and air resistance, contributing to how quickly the motor slows down when not powered. These electrical and mechanical components together form the core of the motor's transfer function in the Laplace domain.

For the robot's kinematic parameters, the wheel radius R=0.025m converts rotational speed into linear speed, while the robot mass M=1kg influences how much force is required to accelerate the robot forward. The wheelbase L=0.25m is the distance between the wheels, critical for calculating turning radius and rotational control. Finally, the moment of inertia about the vertical axis I_z=0.02 kg\m^2 governs the resistance to angular acceleration during rotation, affecting the angular velocity response.

Motor Transfer Function:

The transfer function of a DC motor was derived by analyzing both the electrical and mechanical dynamics of the system. On the electrical side, Kirchhoff's voltage law was applied to the armature circuit, resulting in an equation that included the motor's inductance, resistance, and back electromotive force (emf). Simultaneously, the mechanical behavior was described by relating the torque generated by the motor to its angular velocity, accounting for inertia and viscous friction. The torque was expressed as being directly proportional to the armature current, while the back emf was related to the angular velocity. These equations were then converted into the Laplace domain to facilitate algebraic manipulation. By substituting the mechanical equation into the electrical one and isolating the output angular velocity over the input voltage, the motor's open-loop transfer function was obtained. This function characterizes how the motor responds dynamically to changes in input voltage, with its numerator determined by the torque constant and its denominator reflecting the combined effects of electrical resistance, mechanical damping, and motor inertia.

$$G_{motor}(s) = \frac{K_t}{Js^2 + bs + \frac{K_e K_t}{R_a}}$$

This transfer function is used to model each DC motor's dynamic response. It is derived from the combination of electrical and mechanical dynamics of the motor. The torque constant Kt, the moment of inertia JJJ, the viscous friction coefficient b, the back-emf constant Ke, and the armature resistance Ra are all included. The equation is expressed in the Laplace domain to allow easier simulation and analysis of system response. This function is applied to both the left and right motors independently. Based on the parameters we have calculated, the transfer function is given as:

$$\frac{0.5}{0.5s^2 + 0.5s + 0.2083}$$

Linear Velocity Equation:

$$v(s) = \frac{R}{2M} \cdot G_{motor}(s) \cdot (V_L(s) + V_R(s))$$

The robot's forward or linear velocity is determined by applying equal voltage to both motors. The voltages VL and VR are summed and passed through the motor transfer function. This result is then scaled by the wheel radius R and the robot's mass M. The outcome is the Laplace-transformed linear velocity v(s). This relationship ensures that the translational motion is driven equally by both wheels.

Angular Velocity Equation:

$$\omega(s) = \frac{R}{LI_z} \cdot G_{motor}(s) \cdot (V_R(s) - V_L(s))$$

The angular velocity, or rotational speed, of the robot is produced when a voltage difference is applied between the two wheels. The right and left motor voltages are subtracted, then passed through the same motor transfer function. The result is scaled by the wheel radius R,

the wheelbase L, and the moment of inertia I_z. This determines how quickly the robot rotates around its center.

The two equations present the forward velocity v(s) and angular velocity ω(s) of a differential-drive mobile robot are derived using the DC motor transfer function Gmotor(s). This transfer function models the motor's dynamic response and includes physical parameters such as the motor torque constant Kt, the inertia J, the damping coefficient b, the back electromotive force constant Ke, and the armature resistance Ra. In Laplace domain, it expresses how the angular velocity of the motor shaft changes in response to an applied voltage.

The linear velocity equation v(s) reflects the forward motion of the robot, which results from applying the same voltage to both left and right motors. The equation v(s) shows that the combined voltage inputs are processed through the motor transfer function and then scaled by the robot's physical parameters: the wheel radius R and mass M. This describes how the average action of both motors leads to translational motion of the robot.

The angular velocity equation ω(s) models the robot's rotation about its center. In this case, the difference between the right and left motor voltages generates a torque, which results in angular motion. This torque is again processed through the same motor transfer function and scaled by parameters such as the wheel radius R, the wheelbase L, and the moment of inertia I_z. This describes how voltage imbalances between the wheels lead to the rotation or turning of the robot.

In summary, both v(s) and ω(s) represent system outputs that depend on the motor dynamics governed by Gmotor(s) and the robot's physical characteristics. The use of summation and subtraction of motor voltages allows the modeling of translational and rotational motion, respectively. These equations are fundamental for simulation, control system design, and real-time implementation in robotic systems.

PID controller equation:

$$u(t) = k_p e(t) + k_i \int e(t)\,dt + k_d \frac{de(t)}{dt}$$

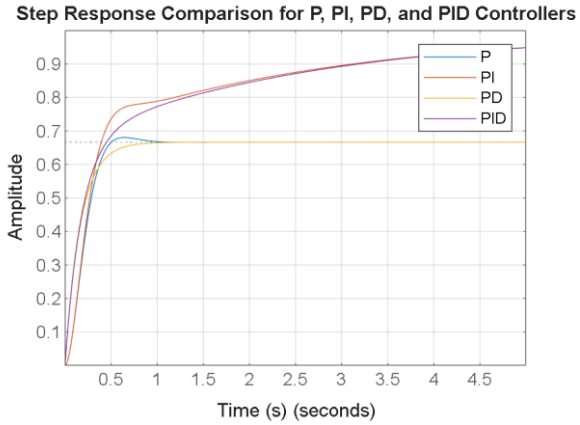
$$e(t) = \text{target position} - \text{current position}$$

The equation shown represents the fundamental control law used in a Proportional-Integral-Derivative (PID) controller. In this form, the control signal u(t) is calculated by summing three terms: the proportional, integral, and derivative responses to the error signal e(t). The error e(t) is defined as the difference between the target position and the current measured position. This discrepancy is continuously monitored and used to correct the system's behavior over time.

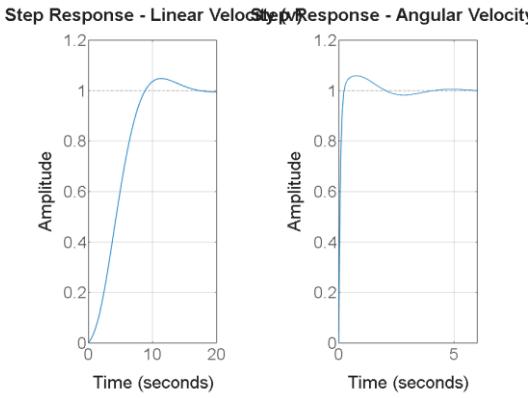
In the proportional term, the current error is multiplied by a gain constant kp. By doing so, a correction is applied that is directly proportional to the size of the error. If a large deviation from the target is detected, a stronger corrective effort is generated. In the integral term, the cumulative sum of past errors is multiplied by the gain ki, which allows for the elimination of steady-state error by addressing small, persistent discrepancies that might not be corrected by the proportional term alone. The derivative term considers the rate at which the error is changing over time. By multiplying the error's derivative by the gain kd, the system's response is dampened to reduce overshooting and improve stability.

This equation is implemented within a closed-loop feedback system, where the actual position is measured using a sensor and fed back into the controller. Based on the continuous comparison between the desired and actual positions, a new control signal is recalculated and sent to the plant—in this case, the motors of the mobile robot. The motors then adjust their behavior accordingly, and this adjustment is again sensed and fed back into the system.

8. MATLAB Simulink Modeling



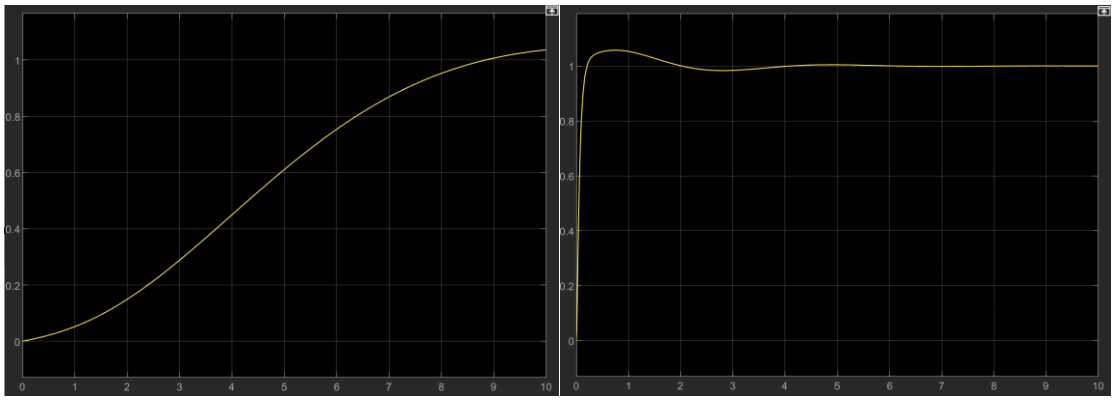
The step responses for the four controllers—P, PI, PD, and PID—were simulated and analyzed. It was observed that the proportional controller produced a fast but incomplete response, with a notable steady-state error. When integral action was introduced in the PI controller, the steady-state error was eliminated, though at the cost of increased overshoot and slower settling. The PD controller, on the other hand, exhibited improved damping and reduced overshoot, but was still unable to correct the steady-state error. Finally, the PID controller demonstrated the most balanced performance, where the transient response was refined, the overshoot was minimal, and the steady-state error was nearly eliminated. These results validated the distinct functional advantages of each control approach.

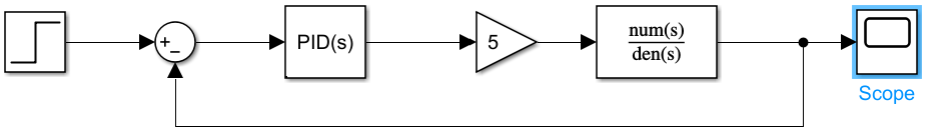


using the PID parameters Kp=4, Ki=8, and Kd=3 demonstrate a significant improvement in the control performance of both linear and angular velocity responses. These gains have led to a system that is faster, more stable, and better damped than previous configurations.

For the linear velocity (v), the step response shows a rise time of approximately 5.77 seconds, which reflects how quickly the robot accelerates to reach its desired forward speed. Compared to earlier simulations where the rise time was longer and the response slower, this configuration achieves a good balance between responsiveness and stability. The overshoot is now around 4.8%, which is relatively mild and indicates the system slightly exceeds the target velocity before settling. The settling time of 14.71 seconds means that the system stabilizes within an acceptable range of the final value in a reasonable duration, with minimal oscillations.

As for the angular velocity (ω), the improvements are even more pronounced. The system exhibits a very fast rise time of just 0.135 seconds, meaning it quickly responds to rotation commands. The overshoot of 5.9% is minimal, showing that the controller effectively dampens the initial surge in rotational speed. The settling time is also short at 1.66 seconds, indicating that the system becomes stable almost immediately after the initial adjustment. The peak value and timing reflect a well-damped system with little to no sustained oscillation.





The Simulink simulations and corresponding step response plots provide a clear visualization of how the PID controller influences both the linear and angular velocity dynamics of the mobile robot. The first graph shows the closed-loop step response of the linear velocity transfer function v(s), which represents the translational motion of the robot. The curve begins at zero and rises smoothly toward its steady-state value, which slightly exceeds one. There is no observable overshoot or oscillation in this response, indicating a highly damped system. This behavior suggests that the PID controller settings applied to the linear velocity model favor stability and minimal overshoot, although at the cost of a longer rise time. The response takes about 9 to 10 seconds to settle fully, which implies that the controller could benefit from slightly more aggressive tuning to improve response speed.

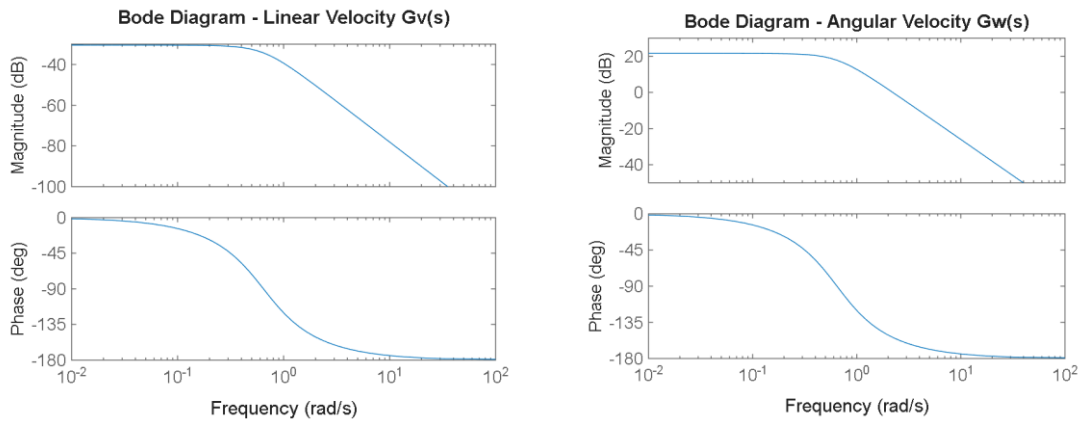
The second graph corresponds to the angular velocity transfer function w(s), which models the robot's rotational motion. Compared to the linear velocity response, the angular velocity step response is much faster, reaching its peak in approximately 1 to 2 seconds. A small overshoot is observed, followed by a quick damping and convergence to the final value. This indicates that the PID controller for angular velocity is tuned more aggressively than the one for linear velocity. The overshoot remains within acceptable limits, and the response shows only minor oscillations, demonstrating that the controller provides a good balance between responsiveness and stability for this subsystem.

The Simulink block diagram used to simulate the closed-loop system. The layout follows a classic negative feedback control structure. A step input is provided as a reference signal, which is compared to the feedback from the system output. The error signal is fed into a PID controller block, which applies proportional, integral, and derivative actions to reduce the error over time. The gain block labeled as “K” in the Simulink model plays a crucial role in scaling the control effort generated by the PID controller before it is applied to the system’s transfer function. In the simulations, this gain was set based on the constants derived from the physical parameters of the robot’s dynamics—specifically the wheel radius R, mass M, wheelbase L, and inertia I_z. For the linear velocity v(s), the gain K=R/2M , while for angular velocity w(s), the gain $K=R/LI_z$. These gains translate the controller’s voltage output into a physically meaningful torque input for each velocity model, ensuring accurate representation of real-world behavior in the simulation. ThE control signal is then scaled using a gain block before entering the transfer function block that represents the plant dynamics (either v(s) or w(s)). The system output is routed back into the summing junction to form a feedback loop, and the overall system response is visualized using a Scope block.

The PID gains used in these simulations were Kp=4, Ki=8, and Kd=3. These values provide moderate control action with enough proportional and derivative terms to ensure responsive yet stable behavior. For the angular velocity system, these gains resulted in a well-balanced

response with a quick rise and minimal oscillation. For the linear velocity system, the same gains led to a more sluggish but overshoot-free response. This difference arises from the inherent dynamics of each transfer function — particularly their numerator values — and suggests that gain tuning may need to be customized for each subsystem to achieve optimal performance.

The closed-loop step response shown in the graph illustrates the behavior of the system under PID control. The output gradually increases from zero and smoothly approaches the desired target value without exhibiting any overshoot or oscillations. This indicates a critically damped or slightly overdamped system, where stability has been achieved without compromising response time significantly. The smooth curve suggests that the proportional, integral, and derivative gains have been tuned effectively to ensure steady convergence to the setpoint. The PID controller was able to drive the system to reach and maintain the desired amplitude efficiently, highlighting its suitability for precise and stable control of the mobile robot.



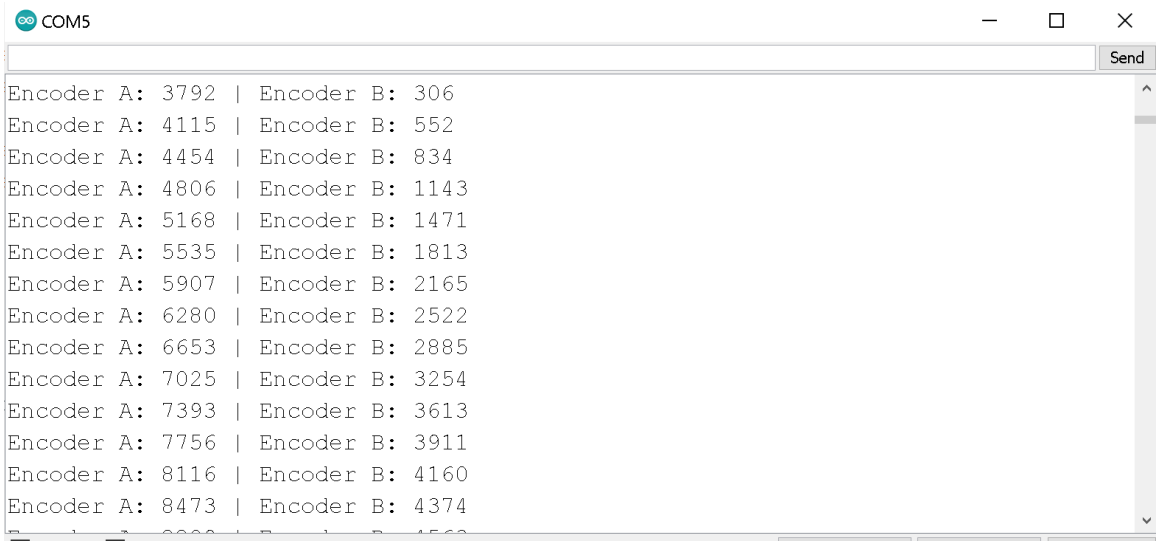
The Bode diagrams provided represent the frequency response characteristics of the system's two main transfer functions: the linear velocity transfer function Gv(s) and the angular velocity transfer function Gw(s). These plots offer critical insights into how the robot responds to varying frequencies in both translational and rotational motion.

For the linear velocity Gv(s) Bode plot, the magnitude begins at a low level and starts decreasing significantly beyond a certain frequency, which identifies the system's bandwidth limit. The phase plot reveals a typical second-order system behavior: a gradual phase drop from 0° towards -180°, indicating increasing lag with higher frequencies. The system is mostly responsive at lower frequencies, which is expected for velocity control where high-frequency components often represent noise and should be attenuated.

In contrast, the angular velocity Gw(s) Bode diagram starts with a much higher initial gain (around +20 dB), indicating a greater sensitivity to input changes compared to the linear velocity case. Similar to Gv(s), the magnitude also drops off at higher frequencies, and the phase falls in a comparable manner. The high initial magnitude in the angular velocity

response reflects the system's faster and more dynamic behavior in terms of rotation, which aligns with the previously observed step responses showing quicker rise and settling times.

9. Results




```
Encoder A: 6099 | Encoder B: 5600 || RPM A: 4311.0 | RPM B: 3687.0
Encoder A: 7384 | Encoder B: 6940 || RPM A: 3840.0 | RPM B: 4011.0
Encoder A: 8229 | Encoder B: 7815 || RPM A: 2523.0 | RPM B: 2613.0
```

The results shown in the images reflect real-time feedback from rotary encoders attached to two DC motors. The encoders provide pulse counts that increase as the motors rotate, indicating movement and allowing for tracking the relative position or speed of each wheel. This data is essential for applications involving precise motor control, such as robotics or mobile platforms, where knowing the rotation progress of each motor helps maintain direction and velocity.

In the second display, RPM (revolutions per minute) values are calculated from the encoder counts. These values give a direct representation of motor speed and are especially useful when implementing closed-loop control systems like PID. By comparing RPMs from both motors, the system can identify mismatches in performance and apply corrective signals to balance them. Overall, these results confirm that the encoder system is functioning correctly and providing the necessary feedback for real-time motor control.

```
// Direction control logic
void moveCar(String dir) {
  if (dir == "FORWARD") {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);   // Motor A forward
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);   // Motor B forward
  }
  else if (dir == "BACKWARD") {
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);   // Motor A backward
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);   // Motor B backward
  }
  else if (dir == "LEFT") {
    digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);  // Motor A backward
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);   // Motor B forward
  }
  else if (dir == "RIGHT") {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);   // Motor A forward
    digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);  // Motor B backward
  }
  else {
    // STOP
    digitalWrite(IN1, LOW);  digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);  digitalWrite(IN4, LOW);
    pwmA = 0;
    pwmB = 0;
```

```
Received: DIR:FORWARD,SPD:18
Encoder A: 387 | Encoder B: 373 || RPM A: 1161.0 | RPM B:
1119.0
```

The provided image displays part of an Arduino program responsible for controlling the movement direction of a two-motor robotic car based on received commands. The moveCar() function takes a string parameter dir and adjusts the logic levels of digital pins connected to the motor driver accordingly. For example, when the direction is "FORWARD", both motors are set to rotate in the forward direction. Similarly, for "BACKWARD", "LEFT", and "RIGHT" commands, the logic switches motor directions to achieve the desired motion. If none of the specified directions are matched, the system enters a stop state, turning off all motor pins and setting the PWM signals to zero.

The second part of the output shows a serial monitor result after a command is received. The system reads and displays the direction (FORWARD) and the speed, alongside the encoder counts and calculated RPMs for both motors. This confirms that the command was successfully interpreted and executed, and the motors responded by rotating, with encoder feedback reflecting their speeds. This setup illustrates a real-time feedback and control loop useful in robotic vehicle applications for precise directional and speed control.

```
if (absPitch > absRoll && absPitch > DEADZONE) {
  if (smoothedPitch > 0) {
    direction = "FORWARD";
    speed = map(smoothedPitch, DEADZONE, 45, 5, 20);
  } else {
    direction = "BACKWARD";
    speed = map(smoothedPitch, -DEADZONE, -45, 5, 20);
  }
} else if (absRoll > absPitch && absRoll > DEADZONE) {
  if (smoothedRoll > 0) {
    direction = "RIGHT";
    speed = map(smoothedRoll, DEADZONE, 45, 5, 20);
  } else {
    direction = "LEFT";
    speed = map(smoothedRoll, -DEADZONE, -45, 5, 20);
  }
}
```

The provided Arduino code snippet is a decision-making block that determines the movement direction and speed of a mobile robot based on pitch and roll angles from an IMU (Inertial Measurement Unit). The logic compares the absolute values of pitch and roll angles to identify the dominant motion axis. If pitch is greater than roll and also exceeds a predefined DEADZONE, the robot is instructed to move forward or backward. A positive pitch leads to a "FORWARD" direction, while a negative pitch triggers "BACKWARD" movement. The map() function is used to scale the smoothed pitch value into an appropriate speed range between 5 and 20.

On the other hand, if roll is greater than pitch and exceeds the DEADZONE, the robot interprets the motion as a command to turn. A positive roll results in a "RIGHT" direction, whereas a negative roll corresponds to "LEFT". Again, the speed is mapped using the smoothed roll value. This structure ensures that the robot only moves when the tilt angle is significant enough to cross the deadzone threshold, helping avoid jittery or unintended movements due to small fluctuations in orientation. The design effectively enables gesture-based control of the robot using head or hand tilts, offering an intuitive human-machine interface.

10. Discussion

This project successfully demonstrated a wireless-controlled mobile robot system operated through tilt input from an IMU sensor. The integration of two Arduino UNO boards enabled clear separation between the transmitter and receiver modules, with the APC220 modules facilitating real-time wireless communication. The MPU9250 sensor provided reliable roll and pitch data to interpret user commands. Directional movement was achieved through logical mapping of tilt orientation, and PWM signals were used to modulate motor speed according to the degree of tilt.

Throughout the implementation, several challenges emerged. One of the primary issues was ensuring consistent wireless transmission between the transmitter and receiver. Occasional packet delays or missed commands were observed, particularly when environmental

interference was present. Another problem appeared during motor calibration — at times, one motor would respond differently from the other due to slight variations in wiring, motor quality, or friction. This was particularly noticeable during forward and backward movement tests, where one motor would lag or halt, requiring repeated adjustments and encoder feedback-based monitoring.

Power supply and current delivery to the motors was another significant challenge. Initially, the battery pack chosen for the robot could not handle the current spikes required during sudden motor activations or directional changes. This led to voltage drops and inconsistent motor behavior. In one case, excessive current draw resulted in overheating and damage to the battery pack, causing a failure that required replacement. To solve this, more robust batteries with higher current ratings were used, and care was taken to monitor the wiring and current distribution to the L298N motor driver. These changes improved the stability of the robot during continuous operation and prevented further power-related failures.

Additionally, the speed control using the map() function sometimes produced erratic behavior at lower tilt angles, due to sensitivity in the sensor readings. Filtering techniques, such as smoothing the pitch and roll values, were implemented to mitigate this effect, though some instability remained, especially near the dead zone threshold.

The system modeling and simulation portion in MATLAB and Simulink gave valuable insights into the behavior of the linear and angular velocity transfer functions. Bode plots and root locus diagrams helped analyze system stability and frequency response, while the PID controller design improved tracking performance. However, in the real system, tuning PID values precisely to match simulation behavior proved difficult due to unmodeled disturbances like wheel slippage, battery voltage drop, and surface irregularities.

A major limitation was the lack of a more robust feedback loop in real time. Although encoder data and RPM values were displayed, they were not yet fully closed-loop regulated in hardware using PID — only monitored. This limits the robot's ability to self-correct in real-world scenarios. Also, reliance on the APC220 modules required line-of-sight conditions for optimal communication, which might not be feasible in more complex environments.

11. Conclusion

This project successfully implemented a wireless-controlled mobile robot system using an IMU sensor for intuitive tilt-based navigation. Through effective integration of components such as Arduino boards, APC220 modules, and a motor driver, the robot was able to respond in real-time to directional commands. Despite facing challenges related to motor synchronization, wireless stability, and power supply issues, the system was refined through calibration and hardware adjustments. Overall, the project demonstrated the feasibility of motion control via body orientation and laid the groundwork for future improvements, including enhanced feedback control and more robust power management.

12. References

Dorf, R. C., & Bishop, R. H. (2016). Modern Control Systems (13th ed.). Pearson.

Nise, N. S. (2014). Control Systems Engineering (7th ed.). Wiley.

Horowitz, Paul, and Winfield Hill. *The Art of Electronics*. 3rd ed., Cambridge University

Ogata, Katsuhiko. *Modern Control Engineering*. 5th ed., Prentice Hall, 2009.